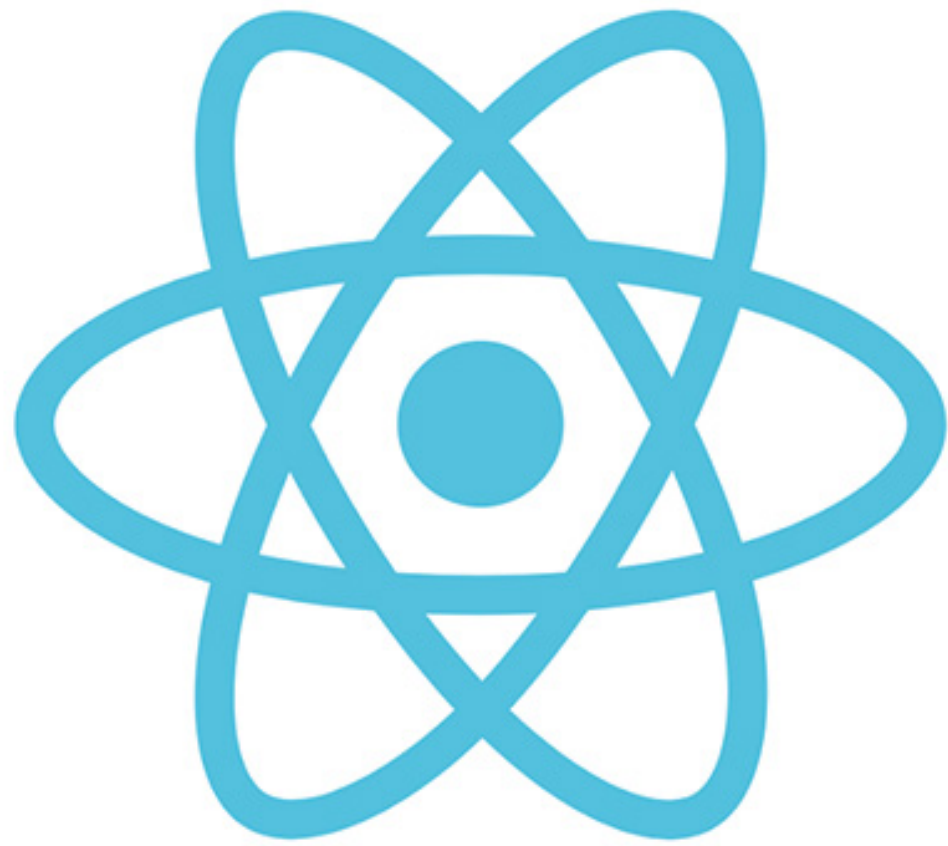


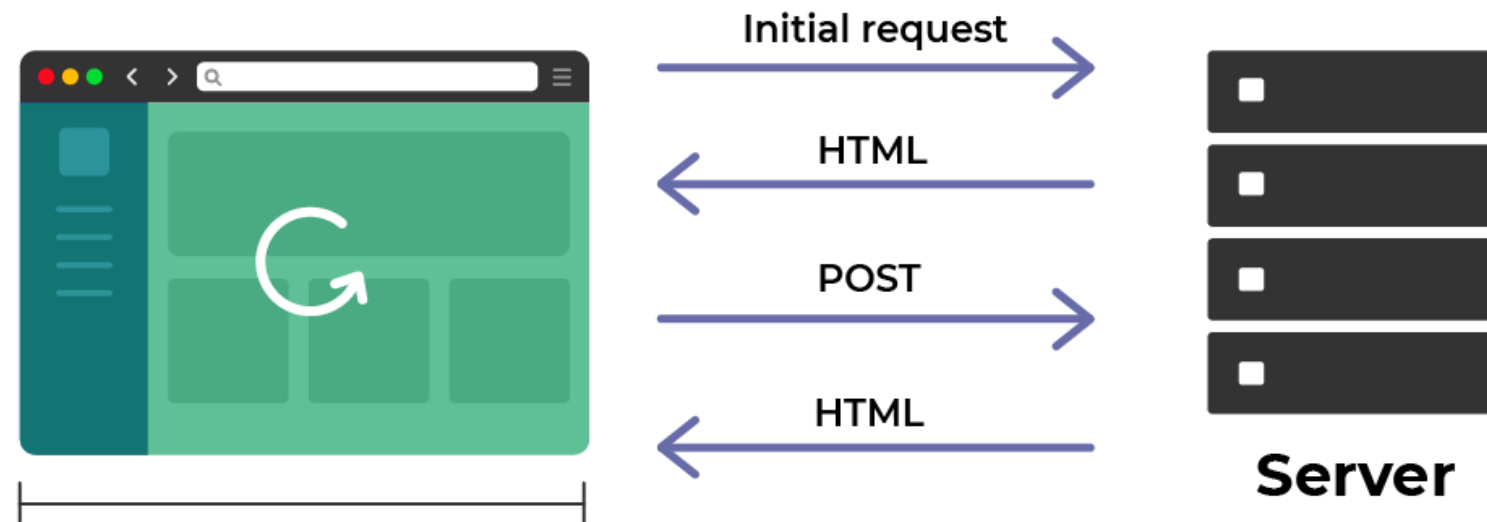


React JS

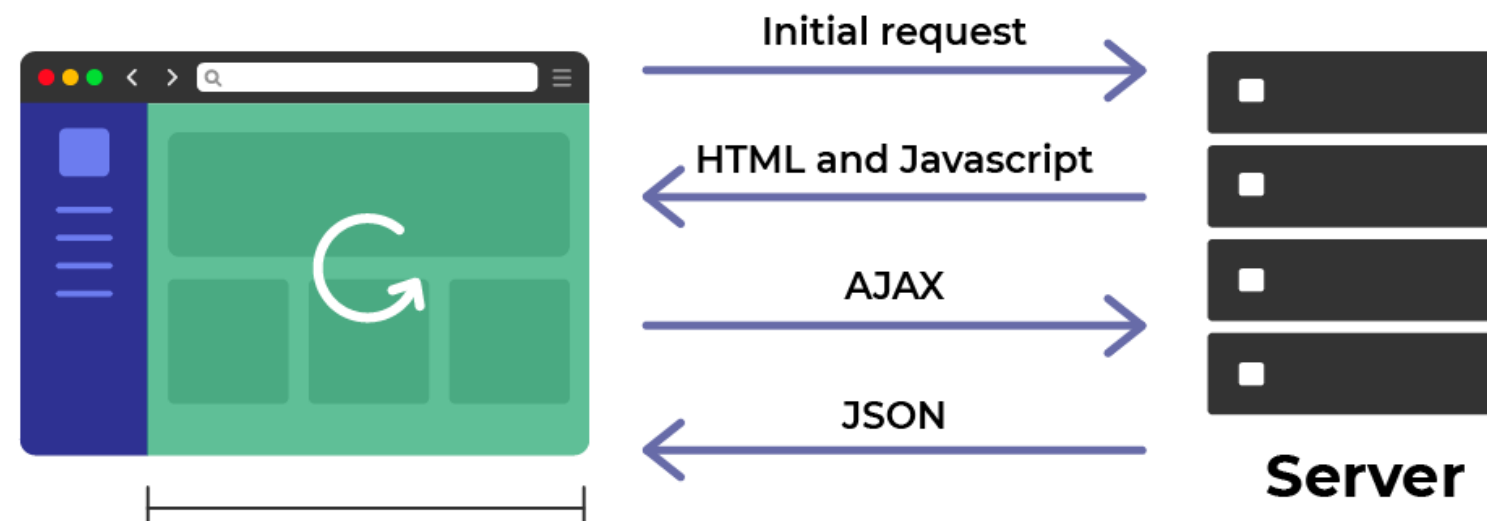
Route

SPA(Single Page App)

Classic website



SPA



React Router 사용하기

```
export default function Header() {  
  return (  
    <>  
      <h1>헤더입니다.</h1>  
    </>  
  );  
}
```

```
export default function Main() {  
  return (  
    <>  
      <h3>안녕하세요. 메인페이지 입니다.</h3>  
    </>  
  );  
}
```

```
export default function Product() {  
  return (  
    <>  
      <h3>상품 페이지입니다.</h3>  
    </>  
  );  
}
```

```
export default function NotFound() {  
  return (  
    <>  
      <h1>404 Error</h1>  
    </>  
  );  
}
```

BrowserRouter 사용

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';  
function App() {  
  return (  
    <div className="App">  
      <BrowserRouter>  
        <Header />  
        <Routes>  
          <Route path="/" element={<Main />} />  
          <Route path="/product/:id" element={<Product />} />  
          <Route path="*" element={<NotFound />} />  
        </Routes>  
      </BrowserRouter>  
    </div>  
  );  
}
```

Link 사용하기

```
import { Link } from 'react-router-dom';
export default function Header() {
  return (
    <>
      <Link to="/">
        <h1>헤더입니다.</h1>
      </Link>
    </>
  );
}
```

```
export default function Main() {
  return (
    <>
      <h3>안녕하세요. 메인페이지 입니다.</h3>
      <Link to="/product/1"> <li>1번상품</li> </Link>
      <Link to="/product/2"> <li>2번상품</li> </Link>
    </>
  );
};
```

createBrowserRouter를 사용

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const router = createBrowserRouter([
  { path: '/', element: (<> <Header /><Main /> </>), },
  { path: '/product/*', element: (<> <Header /><Product /> </>), },
  { path: '*', element: (<> <Header /><NotFound /> </>), },
]);

export default function App() {
  return (
    <div className="App">
      <RouterProvider router={router} />
    </div>
  );
}
```


Layout 사용

```
import { createBrowserRouter, RouterProvider, Outlet } from 'react-router-dom';
const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Main /> },
      { path: 'product/:id', element: <Product /> },
      { path: '*', element: <NotFound /> },
    ],
  },
]);
export default function App() {
  return (
    <div className="App">
      <RouterProvider router={router} />
    </div>
  );
}
```

```
function Layout() {
  return (
    <>
      <Header />
      <Outlet />
    </>
  );
}
```


url에서 데이터 읽기

```
export function Product() {  
  const { id } = useParams();  
  const [searchParams] = useSearchParams();  
  const q = searchParams.get('q');  
  const location = useLocation();  
  return (  
    <>  
    <h3>{id}번 상품 페이지입니다.</h3>  
    <p>검색어 q: {q}</p>  
    <p>pathname: {location.pathname}</p>  
    <p>search: {location.search}</p>  
    <p>hash: {location.hash}</p>  
    </>  
  );  
}
```

합성(Composition)

합성 이란?

합성 예제

Card 1

This is the first card content.

Card 2

This is the second card content.

Click me

Card 3

100 × 100

합성을 사용한 Component

```
import { type ReactNode } from 'react';

type CardProps = {
  children: ReactNode;
};

function Card({ children }: CardProps) {
  const cardStyle: React.CSSProperties = {
    border: '1px solid #ccc',
    borderRadius: '8px',
    padding: '16px',
    margin: '16px',
    backgroundColor: '#f9f9f9',
  };

  return <div style={cardStyle}>{children}</div>;
}
```

합성을 사용한 Component 활용

```
export default function CardList() {  
  return (  
    <div>  
      <h1>합성 예제</h1>  
      <Card>  
        <h2>Card 1</h2>  
        <p>This is the first card content.</p>  
      </Card>  
      <Card>  
        <h2>Card 2</h2>  
        <p>This is the second card content.</p>  
        <button>Click me</button>  
      </Card>  
      <Card>  
        <h2>Card 3</h2>  
          
      </Card>  
    </div>  
  );  
}
```

Form과 입력요소

HTML의 form tag

- HTML <form> 태그는 사용자가 입력한 데이터를 서버로 전송하는 구조를 제공
- 입력 필드와 버튼을 그룹화하여 폼 데이터 제출 및 관리 기능을 수행.

```
<form action="https://example.com/api/join" method="post">
  <!-- 이름 입력 -->
  <label for="name">이름:</label>
  <input type="text" id="name" name="name" required><br>

  <!-- 이메일 입력 -->
  <label for="email">이메일:</label>
  <input type="email" id="email" name="email" required><br>

  <!-- 비밀번호 입력 -->
  <label for="password">비밀번호:</label>
  <input type="password" id="password" name="password" required><br>

  <!-- 제출 버튼 -->
  <input type="submit" value="제출">
</form>
```


React의 form - 제어 컴포넌트

```
export default function ControlledForm() {  
  const [value, setValue] = useState('');  
  
  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {  
    setValue(e.target.value);  
  };  
  
  const handleSubmit = () => {  
    console.log(value);  
  };  
  
  return (  
    <div>  
      <input value={value} onChange={handleChange} />  
      <button onClick={handleSubmit}>전송</button>  
    </div>  
  );  
}
```


React의 form - 비제어 컨트롤로

```
export default function UncontrolledForm() {  
  const inputRef = useRef<HTMLInputElement>(null);  
  
  const handleSubmit = () => {  
    console.log(inputRef.current?.value);  
  };  
  
  return (  
    <div>  
      <input ref={inputRef} />  
      <button onClick={handleSubmit}>전송</button>  
    </div>  
  );  
}
```

React의 form - State로 관리

```
import { useState } from 'react';
export default function StateForm() {
  // 폼으로 취급하는 값을 State로 선언
  const [form, setForm] = useState({
    name: '홍길동', age: 18, });
  // 폼 요소의 변경 사항을 State에 반영
  const handleForm = (e) => {
    setForm({
      ...form,
      [e.target.name]: e.target.value,
    });
  };
  // [보내기] 버튼으로 로그에 메시지 출력하기
  const show = () => {
    console.log(`안녕하세요!!, ${form.name} ( ${form.age}세 ) 님! `);
  };
}
```

이름:

나이:

안녕하세요, 오이영 (20세) 님 !

=>아래에 계속

React의 form - State로 관리

```
return (  
  <form>  
    <div>  
      <label htmlFor="name">이름: </label>  
      <input id="name" name="name" type="text"  
        onChange={handleForm} value={form.name} />  
    </div>  
    <div>  
      <label htmlFor="age">나이:</label>  
      <input id="age" name="age" type="number"  
        onChange={handleForm} value={form.age} />  
    </div>  
    <div>  
      <button type="button" onClick={show}>보내기</button>  
    </div>  
    <p> 안녕하세요, {form.name} ( {form.age}세 ) 님 ! </p>  
  </form>  
)  
;
```

React의 form - 비 제어 컴포넌트

```
import { useRef } from 'react';

export default function StateFormUC() {
  // 리액트 요소에 대한 참조 준비
  const name = useRef(null);
  const age = useRef(null);
  // 요소(참조)를 통해 입력값 준비하기
  const show = () => {
    console.log(`안녕하세요, ${name.current.value}
                (${age.current.value}세) 님!`);
  };
};
```

=>아래에 계속

React의 form - 비 제어 컴포넌트

```
return (  
  <form>  
    { /* 준비된 레퍼런스를 각 요소에 연결 */ }  
    <div>  
      <label htmlFor="name">이름: </label>  
      <input id="name" name="name" type="text"  
        ref={name} defaultValue="홍길동" />  
    </div>  
    <div>  
      <label htmlFor="age">나이: </label>  
      <input id="age" name="age" type="number"  
        ref={age} defaultValue="18" />  
    </div>  
    <div>  
      <button type="button" onClick={show}>보내기</button>  
    </div>  
  </form>  
)  
}
```

제어 컴포넌트 vs 비제어 컴포넌트

항목	제어 컴포넌트	비제어 컴포넌트
값 관리	React State	DOM
Rerender	상태값이 바뀌면 발생	발생하지 않음
검증	쉬움	어려움
사용	일반적	성능최적화 등

React.memo

- 부모 컴포넌트가 리렌더되면 자식 컴포넌트까지 리렌더링됨
- memo를 사용하면 props가 변경되지 않으면 리렌더링 하지 않음
- HOC(Higher Order Component)의 일종

```
export function Child({ value }: { value: number }) {  
  console.log('Child 렌더링');  
  return <div>{value}</div>;  
}
```

```
export const ChildWithMemo = React.memo(function Child({  
  value,  
}: {  
  value: number;  
}) {  
  console.log('ChildWithMemo 렌더링');  
  return <div>{value}</div>;  
});
```


React.memo

```
import { Child, ChildWithMemo } from './components/child';
import { useState } from 'react';

export default function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState('');

  return (
    <div>
      <Child value={count} />
      <ChildWithMemo value={count} />
      <button onClick={() => setCount(count + 1)}>count 증가</button>
      <input value={text} onChange={(e) => setText(e.target.value)} />
    </div>
  );
}
```


React.memo

```
import { Child, ChildWithMemo } from './components/child';
import { useState } from 'react';

export default function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState('');

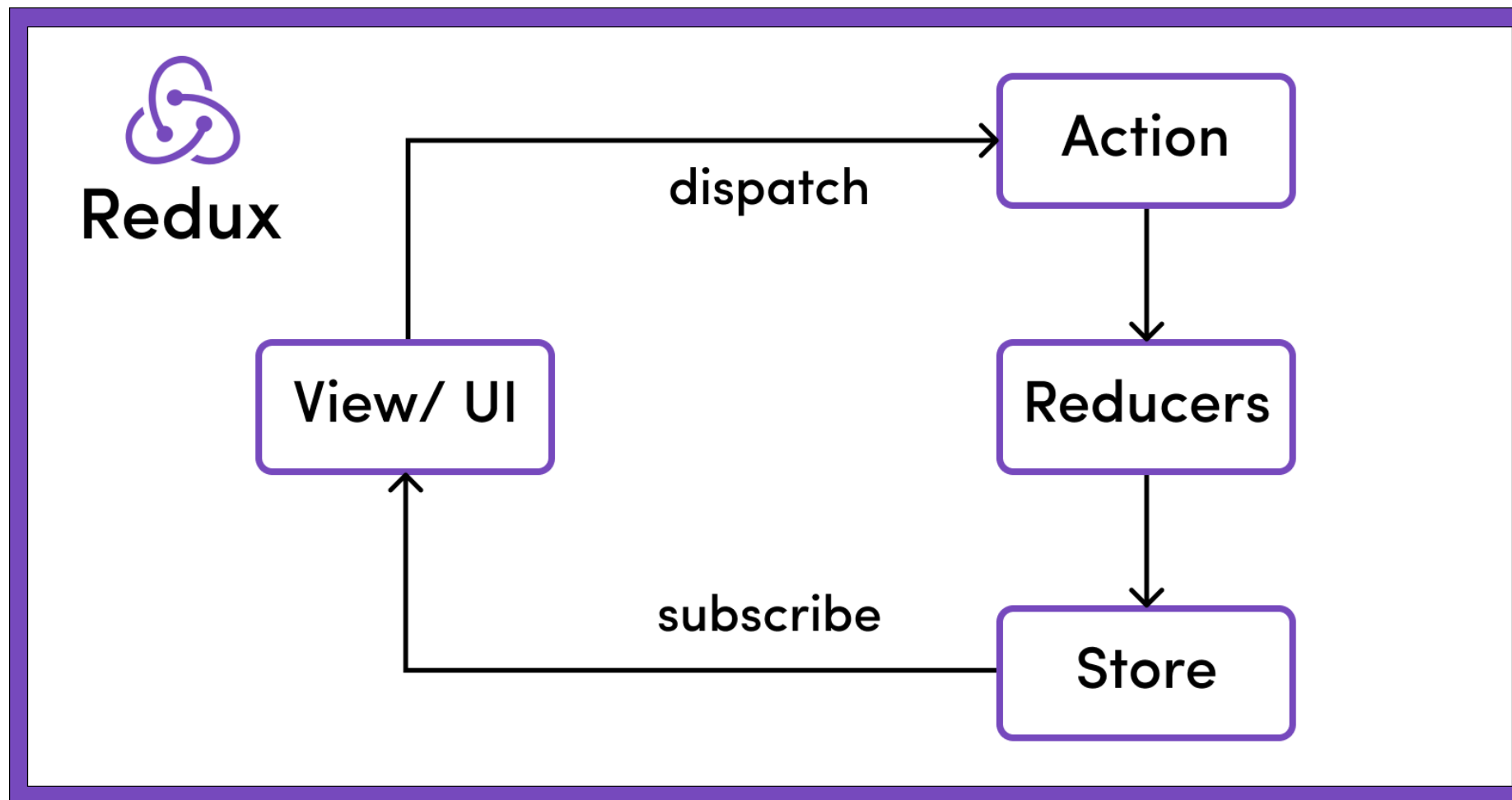
  return (
    <div>
      <Child value={count} />
      <ChildWithMemo value={count} />
      <button onClick={() => setCount(count + 1)}>count 증가</button>
      <input value={text} onChange={(e) => setText(e.target.value)} />
    </div>
  );
}
```



Redux

Redux

- Redux는 애플리케이션의 상태를 한 곳(Store)에서 관리하는 상태 관리 라이브러리
- 모든 상태 변경은 action과 reducer를 통해 예측 가능한 방식으로 이루어짐
- 여러 컴포넌트에서 동일한 데이터를 공유할 때 복잡한 props 전달을 감소
- 상태 흐름이 단방향이라 디버깅과 유지보수가 쉬워짐
- `npm install redux react-redux`



Redux - Action

```
export const INCREASE = 'INCREASE';
```

```
export const DECREASE = 'DECREASE';
```

```
export type Action = { type: 'INCREASE' } | { type: 'DECREASE' };
```

```
export const increase = (): Action => ({  
  type: INCREASE,  
});
```

```
export const decrease = (): Action => ({  
  type: DECREASE,  
});
```

Redux - reducer

```
import { INCREASE, DECREASE, type Action } from './actions';
export type State = { value: number };
const initialState: State = { value: 0,};

export default function reducer(
  state: State = initialState,  action: Action,
): State {
  switch (action.type) {
    case INCREASE:
      return { ...state, value: state.value + 1 };

    case DECREASE:
      return { ...state, value: state.value - 1 };

    default:
      return state;
  }
}
```

Redux - store

```
import { createStore } from 'redux';  
import reducer from './reducer';  
  
export const store = createStore(reducer);
```

Redux - component

```
import { useDispatch, useSelector } from 'react-redux';
import { increase } from './actions';

export default function Counter() {
  const count = useSelector((state: { value: number }) => state.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increase())}>+</button>
    </div>
  );
}
```


Redux - App

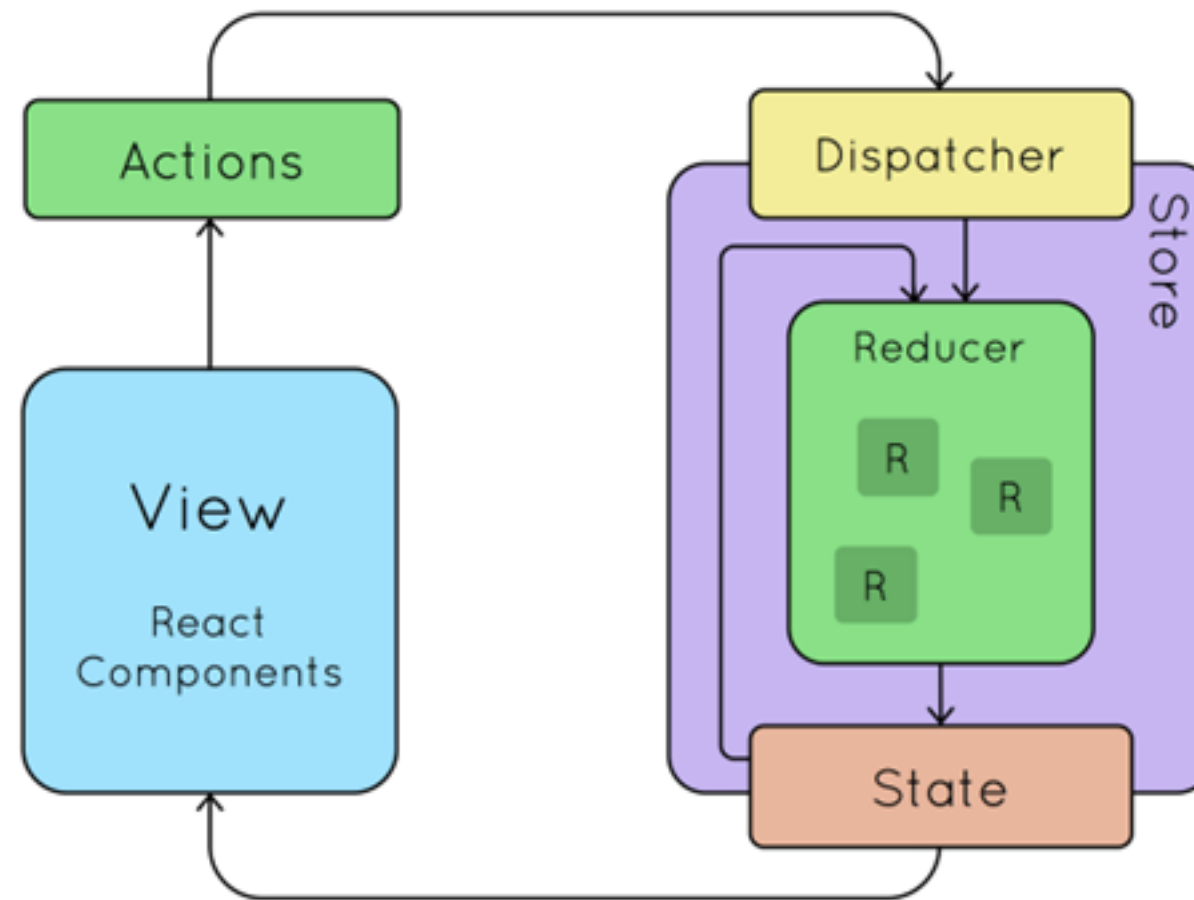
```
import { Provider } from 'react-redux';  
import { store } from './store';  
  
import Counter from './Counter';  
  
export default function App() {  
  return (  
    <Provider store={store}>  
      <Counter />  
      <Counter />  
    </Provider>  
  );  
}
```




Redux Toolkit

Redux Toolkit

- Redux Toolkit은 Redux를 더 쉽고 간결하게 사용하기 위해 공식적으로 제공되는 라이브러리
- 기본 설정과 보일러플레이트 코드를 줄여 개발 생산성 증대
- Immer가 내장되어 있어 상태를 직관적으로 수정 가능



Redux Toolkit - store

```
import { configureStore, createSlice } from '@reduxjs/toolkit';
```

```
const slice = createSlice({  
  name: 'counter',  
  initialState: { value: 0 },  
  reducers: {  
    increase: (state) => {  
      state.value += 1;  
    },  
    decrease: (state) => {  
      state.value -= 1;  
    },  
  },  
});
```

```
export const { increase, decrease } = slice.actions;
```

```
export const store = configureStore({  
  reducer: slice.reducer,  
});
```

Redux Toolkit - component

```
import { useDispatch, useSelector } from 'react-redux';
import { increase } from './store';

export default function Counter() {
  const count = useSelector((state: { value: number }) => state.value);
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increase())}>+</button>
    </div>
  );
}
```

Redux toolkit - combine

```
// counterSlice.ts
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increase: (state) => { state.value += 1 },
    decrease: (state) => { state.value -= 1 },
  },
});

export const { increase, decrease } = counterSlice.actions;
export default counterSlice.reducer;
```

Redux toolkit - combine

```
// userSlice.ts
import { createSlice, type PayloadAction } from '@reduxjs/toolkit';
type UserState = { name: string };
const initialState: UserState = { name: '손흥민' };
const userSlice = createSlice({
  name: 'user',
  initialState,
  reducers: {
    changeName: (state, action: PayloadAction<string>) => {
      state.name = action.payload;
    },
  },
});

export const { changeName } = userSlice.actions;
export default userSlice.reducer;
```

Redux toolkit - combine

```
// store.ts
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './CounterSlice';
import userReducer from './userSlice';
export const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer,
  },
});
// 타입
export type RootState = ReturnType<typeof store.getState>;
```


Redux toolkit - combine

```
import { useDispatch, useSelector } from 'react-redux';
import { increase, decrease } from './CounterSlice';
import { changeName } from './userSlice';
import type { RootState } from './store';
export default function Counter() {
  const count = useSelector((state: RootState) => state.counter.value);
  const name = useSelector((state: RootState) => state.user.name);
  const dispatch = useDispatch();
  return (
    <div>
      <h1>{name}</h1>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increase())}>증가</button>
      <button onClick={() => dispatch(decrease())}>감소</button>
      <button onClick={() => dispatch(changeName('차은우'))}>이름변경</button>
    </div>
  );
}
```


다양한 스타일링 방법

CSS Module

- CSS Module은 컴포넌트 단위로 스타일을 캡슐화하여 클래스 이름 충돌을 방지하는 방식
- 빌드 시 클래스명이 고유한 해시값으로 변환되어 전역 스코프에서 충돌 방지
- 대규모 프로젝트에서 유지보수성과 스타일 재사용성을 높이기 위해 사용

```
/* comp1.module.css */
```

```
.container {  
  background-color: blue;  
  color: white;  
  padding: 10px;  
}
```

```
//com1.tsx
```

```
import styles from './com1.module.css';
```

```
export default function Com1() {  
  return <div className={styles.container}>버튼입니다</div>;  
}
```

```
▼ <body>  
  ▶ <div data-wxt-integrated> ... </div>  
  ▼ <div id="root"> flex  
    <div class="_container_yley4_1">버튼입니다</div> == $0  
  </div>  
  <script type="module" src="/src/main.tsx"></script>  
  <div></div>
```



Material UI

<https://mui.com/>

- Material UI(MUI)는 Google의 Material Design을 기반으로 한 React UI 컴포넌트 라이브러리
- 미리 디자인된 버튼, 카드, 모달 등 다양한 컴포넌트를 제공하여 빠르게 UI를 구축 가능
- 테마 시스템을 통해 일관된 디자인 적용이 가능하며 커스터마이징이 가능
- 접근성과 반응형 설계를 기본 지원해 실무에서 효율적으로 사용 가능

```
npm install @mui/material @emotion/react @emotion/styled
```

```
import { Button } from '@mui/material';
export default function MUISample() {
  const handleClick = () => { alert('버튼 클릭됨!') };
  return (
    <div>
      <Button variant="text">Text</Button>
      <Button variant="contained" onClick={handleClick}>Contained</Button>
      <Button variant="outlined">Outlined</Button>
    </div>
  );
}
```

Tailwind CSS

<https://tailwindcss.com/>

- Tailwind CSS는 미리 정의된 클래스를 조합해 UI를 구성하는 CSS 프레임워크
- 별도의 CSS 파일 없이 HTML/JSX에서 바로 스타일을 적용할 수 있어 개발 속도가 빠름
- 디자인 토큰과 설정을 통해 일관된 스타일 시스템을 구축 가능
- 불필요한 CSS를 제거해 번들 크기를 줄이고 성능을 최적화 가능

```
npm install -D tailwindcss@3 postcss autoprefixer  
npx tailwindcss init -p
```

```
// tailwind.config.js  
export default {  
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],  
  theme: {  
    extend: {},  
  },  
  plugins: [],  
};
```

Tailwind CSS

```
export default function TWCSample() {  
  const handleClick = () => {  
    alert('버튼 클릭됨!');  
  };  
  return (  
    <div className="flex items-center gap-8">  
      <button  
        className="bg-blue-500 text-white px-4 py-2 my-10 rounded"  
        onClick={handleClick}  
      > 버튼 </button>  
      <div className="bg-red-500 text-white w-40 flex justify-center">  
        <span>테스트</span>  
      </div>  
    </div>  
  );  
}
```

```
/* index.css */  
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```